

# Software Development Learning Guide

---

A study guide for someone entering software development, built around the bugs you debugged in these projects ( `LibraryManagementSystem` , `BankTransactionSystem` , `RideHailingSystem` , `JobOrchestrator` , `WalletService` ). Every concept below caused a real failing test in one of them, so you can go back to the code and see it happen.

**How to use it:** read one section, do its "Check yourself" questions without looking, then find the matching bug in the projects and re-read it. Do not try to read it all in one sitting.

## Contents

---

1. Debugging as a skill
  2. Values, references and equality
  3. Collections and the object contracts
  4. Program logic: conditions, boundaries, state
  5. Numbers and money
  6. Concurrency
  7. Databases, transactions and JPA
  8. Web APIs and HTTP
  9. Time, dates and time zones
  10. Testing
  11. Object-oriented design and architecture
  12. Algorithms and data structures
  13. Everyday tooling
  14. Production engineering
  15. A study plan
  16. Glossary
- 

## 1. Debugging as a skill

---

Most of a developer's time goes into reading code and finding out why it does not do what was expected. It is a learnable process, not talent.

### 1.1 The loop

1. **Reproduce** it reliably. If you cannot make it fail on demand, you cannot know you fixed it.
2. **Read the evidence:** the exact error message, the stack trace, the expected-vs-actual values.
3. **Form a hypothesis** ("the total is wrong because the fee is rounded down").
4. **Test the hypothesis** with one small experiment (print a value, run one test, use a debugger).
5. **Fix the cause**, then **re-run everything** (not only the failing test).
6. **Ask why it was possible** (missing test? unclear code?) so it does not return.

## 1.2 Reading a stack trace

```
java.lang.NullPointerException: Cannot invoke "Driver.getId()" because "trip.getDriver()" is
null
    at com.ridehail.service.TripService.getDriverStats(TripService.java:141)    <- first line in
YOUR code
    at Main.scenario16...(Main.java:283)
```

- The top line says **what** happened. The first frame in your own package says **where** it surfaced.
- The bug is often *earlier*: who allowed `getDriver()` to be null? Trace the value backwards.
- Ignore framework frames (Spring, Hibernate, JDK) until your own code does not explain it.

## 1.3 Techniques

| Technique                                   | When it helps                                                  |
|---------------------------------------------|----------------------------------------------------------------|
| Run <b>one</b> test alone                   | Always. Removes noise.                                         |
| Print/log intermediate values               | Quick, works everywhere.                                       |
| <b>Debugger</b> (breakpoint, step, watch)   | When you cannot explain how a value became wrong.              |
| Shrink the input                            | 400 concurrent requests fail? Try 2.                           |
| Binary search the code/history              | "It worked yesterday": <code>git bisect</code> .               |
| Compare with the specification line by line | Logic bugs: a missing clause, a reversed comparison.           |
| Check the mirror-image path                 | <code>deposit</code> has a check, does <code>withdraw</code> ? |
| Rubber-duck explanation                     | Explaining it out loud exposes wrong assumptions.              |

## 1.4 Layered bugs

A bug can hide another (the stale entity made every transfer fail, hiding the deadlock). **Fix one, re-run everything, expect a new failure.** That is normal, not a sign you made it worse.

## 1.5 Never

- Edit the test to make it pass (unless the test itself is provably wrong).
- Fix only the symptom (patch the number the test expects).
- Change five things at once and then run.

**Check yourself:** What is the difference between where an exception is *thrown* and where the bug *is*? Why re-run the whole suite after every fix?

## 2. Values, references and equality

### 2.1 Primitives vs objects (Java)

- Primitives ( `int` , `long` , `double` , `boolean` , `char` ) hold the **value** directly.
- Objects ( `String` , `Integer` , `List` , your own classes) are accessed through a **reference**.
- `==` on primitives compares **values**. `==` on objects compares **whether both refer to the very same object**. To compare contents use `.equals(...)`.

```
String a = "build";
String b = new String("build");
a == b          // false, different objects
a.equals(b)     // true
```

**Bug you met:** `JobRegistry.findByName` used `==` on strings. It passed for literals (Java reuses identical literals) and failed for strings built at runtime.

### 2.2 Wrapper types and the cache trap

```
Integer x = 127, y = 127;   x == y    // true  (small values are cached)
Integer p = 1000, q = 1000; p == q    // false (two separate objects)
```

Never compare `Integer` / `Long` with `==`. Use `.equals`, or unbox ( `int a = ...` ). Same bug class as `Job.hasSamePriorityAs`.

### 2.3 Shared mutable objects (aliasing)

```
public Job(Set<String> deps) { this.deps = deps; }           // keeps the caller's set!
public Job(Set<String> deps) { this.deps = new HashSet<>(deps); } // owns its copy
```

If the caller later changes its set, the job changes silently. Copy on the way **in** and out ( `List.copyOf` , `new ArrayList<>(x)` ), or expose read-only views ( `Collections.unmodifiableList` ). **Bug you met:** `Member.getLoanHistory()` returned the live internal list; a caller could `clear()` it.

### 2.4 Immutability

An object that cannot change after construction ( `String` , `LocalDate` , `record` s) is easy to reason about and automatically thread-safe. Prefer `final` fields and records when you can.

### 2.5 Null

`null` means "no object". Calling a method on it throws `NullPointerException`.

- Decide, per method, whether `null` is allowed; document it.
- Return an empty list, not `null`. Use `Optional<T>` for "maybe absent" results.
- Validate inputs at the boundary of your code, not everywhere.

**Check yourself:** Why does `"a" == "a"` sometimes work and `new String("a") == "a"` not? What is a defensive copy and where do you make it?

## 3. Collections and the object contracts

### 3.1 Choosing a collection

| Need                  | Use                                                     | Notes                                                                       |
|-----------------------|---------------------------------------------------------|-----------------------------------------------------------------------------|
| Ordered, index access | <code>ArrayList</code>                                  | Fast reads, slow middle inserts.                                            |
| Fast lookup by key    | <code>HashMap</code>                                    | Needs correct <code>equals</code> / <code>hashCode</code> .                 |
| Keep insertion order  | <code>LinkedHashMap</code> / <code>LinkedHashSet</code> | Also the basis of an LRU cache.                                             |
| Sorted                | <code>TreeMap</code> / <code>TreeSet</code>             | Needs a consistent <code>compareTo</code> / <code>Comparator</code> .       |
| Priority order        | <code>PriorityQueue</code>                              | Comparator decides "first".                                                 |
| No duplicates         | <code>Set</code>                                        | Duplicate = <code>equals</code> (Hash) or <code>compare == 0</code> (Tree). |

### 3.2 The equals / hashCode contract

1. If `a.equals(b)` then `a.hashCode() == b.hashCode()`.
2. Fields used in `hashCode` must not change while the object sits in a hash collection.

```
// BROKEN: hashCode uses `rating`, equals uses only `id`, and rating changes over time.
public int hashCode() { return Objects.hash(id, rating); }
```

After `driver.setRating(...)` the object hashes to a different bucket and `set.contains(driver)` says `false`. **Bug you met:** `Driver.hashCode` (`RideHailingSystem`). Rule: base both on the same **immutable identity** (usually `id`).

### 3.3 compareTo must be consistent with equals

`TreeSet` treats `compareTo == 0` as "the same element" and silently drops the second one.

```
public int compareTo(Job o) { return Integer.compare(o.priority, priority); } // ties =
"equal"!
```

Two different jobs with the same priority: one disappears. Add a tie-breaker (`... , id`). **Bug you met:** `Job.compareTo` and `JobReport.sortedByPriority`.

### 3.4 Comparators

- Never compare with subtraction: `b.priority - a.priority` overflows for extreme values (`Integer.MAX_VALUE - (-5)` wraps negative). Use `Integer.compare(b.priority, a.priority)`.
- Build multi-key sorts with `Comparator.comparing(...).thenComparing(...)`.

### 3.5 Iterating while modifying

```
for (Trip t : trips) { if (stale(t)) trips.remove(t); } // ConcurrentModificationException
```

Use `Iterator.remove()` or `list.removeIf(...)`. **Bug you met:** `expireStaleRequests`.

### 3.6 Streams and collectors

`Collectors.toMap(key, value)` throws `IllegalStateException` on duplicate keys. When keys can repeat use `groupingBy(...)` or supply a merge function.

### 3.7 An LRU cache in five lines

```
new LinkedHashMap<K,V>(16, 0.75f, true /* accessOrder */) {  
    protected boolean removeEldestEntry(Map.Entry<K,V> e) { return size() > capacity; }  
};
```

`accessOrder=true` moves an entry to the end on every read. With `false` it is a FIFO cache. **Bug you met:** `ResultCache` was configured `false`.

**Check yourself:** Why can a `HashSet` "lose" an object? Why does a `TreeSet` drop elements? What does `removeIf` do that a `for-each` loop cannot?

## 4. Program logic: conditions, boundaries, state

### 4.1 Boundary conditions

The most common bug is off by one at an edge. Always test **below, at, and above** the limit.

- `>` vs `>=`: "up to 500,000" allowed means `used + amount > limit` is the rejection test.
- Half-open intervals: `[start, end)`. Two windows `9-10` and `10-11` **do not overlap**.
- "Exactly 24 hours old has expired": is the boundary inclusive or exclusive? Write it down.

### 4.2 Interval overlap

Two intervals overlap **iff** each starts before the other ends:

```
a.start.isBefore(b.end) && b.start.isBefore(a.end) // AND, not OR
```

**Bug you met:** `AvailabilityWindow.overlaps` used `||`.

## 4.3 Operator precedence

`&&` binds tighter than `||`.

```
frozen || closed && amount > 1000    // means: frozen || (closed && amount > 1000)
(frozen || closed) && amount > 1000  // what you might have meant
```

When mixing them, add parentheses even if not strictly needed. **Bug you met:** the closed-account withdrawal check (BankTransactionSystem).

## 4.4 "any" vs "all"

`anyMatch` vs `allMatch`: a job is *ready* when **all** dependencies are complete, not when any is.

## 4.5 State machines

Model lifecycles explicitly: a set of states and the **legal transitions** between them.

```
ACTIVE <-> FROZEN -> CLOSED (terminal)
REQUESTED -> ACCEPTED -> IN_PROGRESS -> COMPLETED (or -> CANCELLED)
```

- Check the *current* state before every transition (whitelist the legal ones; do not blacklist).
- A condition that can never be true ( `refunded > amount` when the cap prevents it) is a red flag.

## 4.6 Propagation and recursion

"Skip everything that depends on the failed job" means **transitively**, not only direct children. Recursion: base case, then apply the same rule to each child. Track visited nodes to avoid infinite loops.

## 4.7 Exceptions

- **Checked** ( `Exception` ): the compiler forces handling. **Unchecked** ( `RuntimeException` ): bugs and business errors you may choose to let propagate.
- **Never swallow:** `catch (Exception e) { }` hides failures. At minimum log; usually report or rethrow.
- Do not use exceptions for normal flow; do not catch what you cannot handle.
- `finally` always runs; a `return` inside `finally` overrides a thrown exception.
- Design errors with **stable codes** ( `INSUFFICIENT_FUNDS` ) that callers can rely on.

## 4.8 Validate at the boundary, protect in the domain

Validate untrusted input where it enters (HTTP layer: `400` ). Enforce business invariants in the domain object as well (defence in depth), so no caller can break them. The database is the last net, not the user interface.

**Check yourself:** Write the rejection test for "daily limit 500 inclusive". Draw the state diagram for a wallet. When is `catch (Exception e) { }` acceptable? (Almost never.)

## 5. Numbers and money

### 5.1 Integer overflow

Integers wrap around silently at their maximum.

```
Integer.MAX_VALUE + 1    // -2147483648
Long.MAX_VALUE  + 1      // -9223372036854775808
```

Use `Math.addExact` / `multiplyExact` (throws `ArithmeticException`), a wider type, or check first. Also beware `1 << n` (shift counts wrap: `1 << 32 == 1`) and computing then capping (`min(a*b, cap)` overflows *before* the cap). **Bugs you met:** backoff delay, priority comparator, wallet credit.

### 5.2 Integer division

```
int a = 3, b = 2;
double r = a / b;           // 1.0, division happened on ints first
double r = (double) a / b; // 1.5
```

**Bug you met:** the surge-pricing ratio and `totalPages = total / size` (should round **up**).

### 5.3 Floating point is not exact

`0.1 + 0.2 != 0.3`. `double` cannot represent most decimal fractions exactly. **Never use `double` for money.**

### 5.4 Money done right

- Store amounts as **integers in minor units** (cents): `1050` = 10.50.
- For calculations with fractions (fees, interest) use `BigDecimal` with an explicit `RoundingMode`.
- Build `BigDecimal` from a **string** or `BigDecimal.valueOf(double)`, **never** `new BigDecimal(double)`: `new BigDecimal(1.005)` is `1.0049999999999999...`, so it rounds to 1.00 instead of 1.01.
- Compare with `compareTo`, not `equals` (`2.0` vs `2.00` are not `equals`).

### 5.5 Rounding modes

| Mode                              | 5.5 | 2.5 | -2.5 |
|-----------------------------------|-----|-----|------|
| <code>HALF_UP</code>              | 6   | 3   | -3   |
| <code>HALF_EVEN</code> (banker's) | 6   | 2   | -2   |
| <code>DOWN</code> (truncate)      | 5   | 2   | -2   |

Truncating with `(int)` or `RoundingMode.DOWN` under-charges by up to one unit every time. Pick one mode deliberately and document it.

## 5.6 Percentages and basis points

0.5% = 50 basis points (1 bp = 0.01%). Calculate `amount * bps / 10_000` with `BigDecimal` and round once, at the end.

**Check yourself:** Why is `(int)(4.9965*100)/100.0` wrong for money? How would you add two `long` amounts safely? What is `7 / 2` in Java, and `7 / 2.0`?

## 6. Concurrency

Modern services handle many requests at once. Most serious bugs appear only under concurrency.

### 6.1 Threads and shared state

A **thread** is an independent line of execution. Threads that touch the same data can interleave their steps in any order.

### 6.2 Race conditions

`count++` is **three** steps: read, add, write.

| Step | Thread A   | Thread B   | count                      |
|------|------------|------------|----------------------------|
| 1    | reads 100  |            | 100                        |
| 2    |            | reads 100  | 100                        |
| 3    | writes 101 |            | 101                        |
| 4    |            | writes 101 | <b>101</b> (should be 102) |

That is a **lost update**. **Bug you met:** `JobMetrics` counter.

### 6.3 Making code thread-safe

| Tool                                                               | Use for                          |
|--------------------------------------------------------------------|----------------------------------|
| <code>AtomicInteger/AtomicLong</code>                              | a single counter/flag            |
| <code>synchronized</code> / <code>ReentrantLock</code>             | protect a block of several steps |
| <code>ConcurrentHashMap</code> , <code>CopyOnWriteArrayList</code> | shared collections               |
| Immutable objects                                                  | share freely, no locks needed    |
| Confinement                                                        | keep data inside one thread      |

Prefer **not sharing** mutable state at all.

### 6.4 Check-then-act

```
if (balance >= amount) { balance -= amount; } // another thread can act between the two lines
```

The check and the action must be **atomic** (one lock, one transaction, one atomic operation). **Bug you met:** refunds (cumulative cap checked, then applied without a lock).

## 6.5 Deadlock

Deadlock needs a **cycle** of waiting:

```
Transfer A->B: locks A, then wants B
Transfer B->A: locks B, then wants A      -> both wait forever
```

**Fix: lock ordering.** Always acquire locks in one global order (e.g. ascending id), regardless of which the caller listed first. **Bug you met:** `TransferService.lockAll`.

## 6.6 Visibility and reordering (the deeper reason)

Without synchronization one thread may not see another's writes, and the compiler/CPU may reorder operations. `volatile`, locks and atomics establish ordering ("happens-before"). Know that the problem exists even when your tests pass.

## 6.7 Thread pools and thread-locals

Servers reuse threads. Anything stored per thread (a logging context, `MDC`, `ThreadLocal`) **must be cleared in a `finally`**, or it leaks into the next request. **Bug you met:** correlation-id filter.

## 6.8 Testing concurrency

Start many threads at the same instant ( `CountDownLatch` ), run enough iterations for the rare interleaving to show up (400 opposing transfers, not 4), assert **invariants** (total money conserved, no negative balance), and repeat the run several times. A green run does not prove absence.

**Check yourself:** Why is `count++` unsafe? Explain lock ordering with two threads and two locks. What must always be in a `finally` after `MDC.put` ?

---

# 7. Databases, transactions and JPA

## 7.1 Relational basics

Tables, rows, primary key (unique identity), foreign key (reference to another row), indexes (fast lookup), constraints ( `NOT NULL`, `UNIQUE`, `CHECK` ). Learn `SELECT`, `WHERE`, `JOIN`, `GROUP BY`, `ORDER BY`, `INSERT/UPDATE/DELETE` first; everything else builds on them.

## 7.2 Constraints are a safety net

`CHECK (balance_minor >= 0)` and `UNIQUE (owner_id, currency)` protect data even if the application has a bug. But do not rely on them for user-facing behaviour: the app should check first and return a clear error ( `WALLET_ALREADY_EXISTS` ), with the constraint as the race-proof backstop.

## 7.3 Transactions and ACID

A **transaction** groups statements into one unit.

- **Atomic**: all or nothing. **Consistent**: constraints hold. **Isolated**: concurrent transactions do not see each other's half-finished work. **Durable**: committed data survives a crash.
- A failed transfer must roll back **everything** (debit, credit, ledger lines, transfer row).

## 7.4 Isolation levels and locking

- Default is usually **READ COMMITTED**: you see only committed data, but rows can change between your reads.
- **Pessimistic locking** ( `SELECT ... FOR UPDATE` ): take the row lock first, then work. Simple and safe for hot rows; risk of waiting and deadlock (lock in a fixed order).
- **Optimistic locking** ( `@Version` ): no lock; the `UPDATE ... WHERE version = ?` fails if someone else changed the row, and you retry. Great when conflicts are rare.
- A lost update needs one of these. Reading and writing without either is a bug.

## 7.5 Migrations

Schema changes are code: versioned SQL scripts (Flyway) applied in order and never edited afterwards. Set Hibernate to `validate` (not `update` ) so a mismatch fails fast.

## 7.6 The ORM and the persistence context (JPA/Hibernate)

An ORM maps rows to objects. The important idea: within a transaction Hibernate keeps a **persistence context**: one object per row, cached.

- **Dirty checking**: you change an entity's fields and Hibernate writes an `UPDATE` at **flush** (usually at commit). You do not call `save` for updates of managed entities.
- **First-level cache**: a query returning a row that is already in the context gives you the **existing object and ignores the fresh column values**. *So never load an entity with an ordinary read and then lock it later in the same transaction: you would work on stale state.* (**Bug you met**: the fee wallet. Fix: fetch only the id first.)
- `save()` on an entity with an assigned id and a primitive `@Version` may do a `merge` . Use a wrapper `Long version` , or implement `Persistable` .

## 7.7 @Transactional details that bite

- It works through a **proxy**: calling a `@Transactional` method from **another method of the same class** bypasses it. Call through another bean.
- `readOnly = true` is a hint that turns off flushing in Hibernate: writes in it are **silently lost**. (**Bug you met**: `changeStatus` .)
- By default only **unchecked** exceptions roll back.
- Keep transactions short and put the whole business operation (money movement + idempotency record) inside **one**.

## 7.8 Other ORM pitfalls to learn

- **N+1 queries**: loading a list, then one query per element. Use join fetch / batch size.

- **Lazy loading** outside a transaction throws `LazyInitializationException`.
- **Entities are not DTOs:** do not expose them directly in APIs.

**Check yourself:** What does `readOnly = true` do to writes? Why does a query inside a transaction sometimes not reflect the latest DB row? Optimistic vs pessimistic: when would you choose each?

## 8. Web APIs and HTTP

### 8.1 HTTP basics

| Method | Meaning         | Idempotent?                |
|--------|-----------------|----------------------------|
| GET    | read            | yes, safe                  |
| POST   | create / action | no (unless you make it so) |
| PUT    | replace         | yes                        |
| PATCH  | partial update  | usually no                 |
| DELETE | remove          | yes                        |

### 8.2 Status codes you must know

200 OK, 201 Created (+ `Location`), 204 No Content, 400 invalid request, 401/403 unauthenticated/forbidden, 404 not found, 409 conflict, 415 unsupported media, 422 valid but violates a business rule, 429 too many requests, 500 bug/unexpected, 503 unavailable. A validation failure must be 400, never 500. A 500 means *our* bug.

### 8.3 Designing errors

Use a consistent format (RFC 9457 `application/problem+json`): `status`, `title`, `detail`, plus a stable machine-readable `code` and a `correlationId`. Never leak stack traces or internals.

### 8.4 Idempotency (a key production concept)

Networks fail and clients retry. A retried `POST /transfers` must **not move money twice**.

- Client sends `Idempotency-Key: <uuid>`. Server stores (key, request fingerprint, response).
- Same key + **same** request: replay the stored response (same status code) without redoing the work.
- Same key + **different** request: reject ( 422 ). So the fingerprint must cover the **whole** request.
- Reserve the key **in the same transaction** as the operation, so two concurrent duplicates cannot both run.
- Do not cache failures: a rolled-back attempt leaves no record, so the client can retry.

**Bugs you met:** fingerprint hashed only the request *type*; replay returned 200 not 201.

## 8.5 Validation

Use annotations ( `@NotBlank` , `@Positive` , `@Size` , `@Pattern` , `@Max` ) on request DTOs. Bound everything: page sizes, string lengths, amounts. Unbounded input is a denial-of-service bug.

## 8.6 Pagination

Parameters `page` and `size` ; response has `items` , `totalElements` , `totalPages` (**round up**: `ceil(total / size)` ), stable **sort order** (newest first, tie-break by id). Cap `size` .

## 8.7 DTOs and layering

Controller (HTTP <-> DTO) -> service (business rules, transactions) -> repository (database). Keep HTTP concerns out of services and business rules out of controllers.

**Check yourself:** 200 vs 201 vs 204 ? 400 vs 422 ? Why must an idempotency fingerprint include the request body? How many pages for 25 items at size 10?

# 9. Time, dates and time zones

## 9.1 The right types (Java `java.time` )

| Type                                                | Meaning                                            |
|-----------------------------------------------------|----------------------------------------------------|
| <code>Instant</code>                                | a point on the global timeline (store this)        |
| <code>LocalDate</code> / <code>LocalDateTime</code> | calendar date / date-time <b>without</b> a zone    |
| <code>ZonedDateTime</code>                          | date-time <b>with</b> a zone (display, scheduling) |
| <code>Duration</code> / <code>Period</code>         | exact time span / calendar span                    |

Store and compare **instants in UTC**. Convert to a zone only for display or calendar logic.

## 9.2 Daylight saving time

"Tomorrow at 09:00" is **not** "now + 24 hours": on the spring-forward day a day is 23 hours long.

```
lastRun.plusHours(24)    // drifts across DST -> 10:00 local
lastRun.plusDays(1)      // same wall-clock time -> 09:00 local
```

**Bug you met:** `DailySchedule` . Use calendar arithmetic ( `plusDays` ) for calendar rules.

## 9.3 Inject the clock

Never call `Instant.now()` / `LocalDate.now()` inside business logic. Take a `java.time.Clock` as a dependency. Tests can then move time ("24 hours later") without sleeping. **Bug you met:** one ledger line used `Instant.now()` , ignoring the injected clock.

## 9.4 Windows and boundaries

Sliding/rolling windows: decide if the edge is inclusive or exclusive and test exactly at the edge.

**Check yourself:** Why store `Instant` and not `LocalDateTime` ? How long is a day on the DST change? Why inject a `Clock` ?

---

## 10. Testing

---

### 10.1 Kinds of tests

- **Unit:** one class, no I/O, milliseconds ( `FeePolicyTest` , `WalletTest` ).
- **Integration:** several parts with a real database and HTTP layer ( `WalletApiTest` , `MockMvc` ).
- **Concurrency/load:** many threads on the same data.
- **End-to-end:** the deployed system.

Many fast unit tests, fewer integration tests, few end-to-end (the "test pyramid").

### 10.2 What makes a good test

- **Arrange, act, assert** (one behaviour per test, named for the behaviour: `aReplayedTransferDoesNotChargeTwice` ).
- **Deterministic:** no real time, no random data that changes results.
- **Independent:** no test may depend on another; unique data per test.
- Test **behaviour and invariants** ("balance equals sum of ledger") rather than implementation details.
- Cover **boundaries and failure paths**, not just the happy path.

### 10.3 Tests as specification

A failing test says what the system is *supposed* to do. Read tests to learn requirements. A test that passes only for the exact values it uses can hide a bug ( `999.30` happened to round right by luck). Try other inputs.

### 10.4 Flaky tests

A test that passes and fails on the same code depends on time, order, randomness or concurrency. Never "re-run until green": find the shared state. For race conditions run repeatedly.

### 10.5 Mocks vs real

Mocks isolate a unit but can drift from reality. For databases prefer a real one (H2, embedded PostgreSQL) so locking and SQL behaviour are genuine (**H2 and PostgreSQL can disagree**).

### 10.6 TDD in one line

Write a failing test, make it pass, clean up. It forces you to think about behaviour first.

**Check yourself:** Why is `assertTrue(x)` a weak assertion? What makes a test flaky? Why run the same tests on two databases?

---

## 11. Object-oriented design and architecture

---

### 11.1 Encapsulation

Keep fields private; expose behaviour, not raw state. `Wallet.debit(amount)` enforces the rules; a public `setBalance` would let anyone break them. Put the rule where the data lives.

### 11.2 Inheritance and polymorphism

```
abstract class Vehicle { abstract double calculateFare(double miles, double minutes); }  
class PremiumVehicle extends Vehicle { ... }
```

- A subclass must honour the parent's contract (Liskov): a `Premium` fare must still include the base fare.
- `super(...)` runs first; a local variable with the same name as a field **does not** assign the field ( `int capacity = seatCount;` created a throwaway local).
- Overriding needs the exact same signature ( `@Override` makes the compiler check it).
- Prefer **composition** ("has a") over deep inheritance.

### 11.3 SOLID in plain words

- **S**ingle responsibility: one reason to change per class.
- **O**pen/closed: extend behaviour without editing tested code.
- **L**iskov substitution: subtypes behave like their parent.
- **I**nterface segregation: small, focused interfaces.
- **D**ependency inversion: depend on abstractions (inject a `Clock`, a repository) so you can swap/test them.

### 11.4 Layered architecture

`web -> service -> domain + repository`. Dependencies point one way. Business rules live in the domain/service, not in controllers or SQL. Small, named policy classes ( `FeePolicy`, `LimitPolicy` ) make rules easy to test and change.

### 11.5 Design for invariants

Write down what must **always** be true and enforce it in one place:

- balance never negative; refunds never exceed the transfer;
- ledger entries of a transfer sum to zero; `balance == sum of its entries`.

Then write tests that assert the invariants after every kind of operation, including under load.

### 11.6 Copy-paste is a bug factory

Two similar blocks ( `deposit` / `withdraw` ) drift apart. Extract shared checks, or review symmetric paths side by side.

**Check yourself:** Why is a public setter on a balance dangerous? What does Liskov require of a subclass?

## 12. Algorithms and data structures

---

### 12.1 Big-O (how cost grows)

$O(1)$  constant,  $O(\log n)$  binary search,  $O(n)$  one pass,  $O(n \log n)$  good sorting,  $O(n^2)$  nested loops,  $O(2^n)$  try-every-subset. Know it well enough to say "this is quadratic, it will not scale".

### 12.2 Structures to master

Array/list, stack, queue, deque, hash map/set, heap (priority queue), tree, graph. Know the cost of each operation and when to choose which.

### 12.3 Recursion and backtracking

Every recursive function has: a **base case**, and a step that moves toward it. Backtracking explores choices: for each item, **try including it** and **try skipping it**, then keep the best. **Bug you met:** the ride-pool matcher took the first item that fit and never tried skipping it, so it missed the better combination `[2, 2]` versus `[3]`. Greedy is not always optimal.

### 12.4 Graphs and dependency ordering

Jobs with dependencies form a directed graph.

- **Topological sort** (DFS): visit dependencies first, then the node.
- **Cycle detection** needs **two** markers: "in progress" (on the current path) and "done". Using one visited set reports a shared dependency (diamond `a->b, a->c, b->d, c->d`) as a cycle.

### 12.5 Sliding window and rate limiting

Keep timestamps in a deque; drop those older than the window; allow while `size < max`. Decide whether "exactly one window old" is expired. Same idea for the rolling daily withdrawal limit.

### 12.6 Backoff

Retry with exponentially increasing, capped delay plus jitter: `min(base * 2^(n-1), cap)`. Beware overflow of  $2^n$  for large  $n$  (clamp the exponent).

**Check yourself:** Draw the recursion tree for `[3,2,2]`, capacity 4. Why does one `visited` set misreport a diamond as a cycle? What is the cost of `contains` on an `ArrayList` vs `HashSet`?

---

## 13. Everyday tooling

---

### 13.1 Git (learn this first, you will use it daily)

```
git status ; git diff ; git add <files> ; git commit -m "why, not what"
git switch -c feature/x          # branch
git pull --rebase ; git push    # share
git log --oneline --graph ; git bisect  # history and bug hunting
```

Small commits with clear messages. Never commit secrets ( `.env` , keys). Review your own diff before pushing. Undo safely with `git restore` / `git revert` ; be careful with `reset --hard` and force-push.

### 13.2 The terminal

Navigate ( `cd` , `ls` ), search ( `grep` ), view ( `cat` , `less` ), pipe ( `|` ), redirect ( `>` , `>>` , `2>&1` ). Save output: `./mvnw test > out.txt 2>&1` or `... 2>&1 | tee out.txt` . Windows PowerShell: `*>` .

### 13.3 Build tools

Maven ( `pom.xml` ): dependencies, plugins, lifecycle ( `compile` , `test` , `package` ). Use the wrapper ( `./mvnw` ) so everyone builds with the same version. Read dependency errors from the **bottom** up.

### 13.4 The debugger

Set a breakpoint, run in debug mode, step over/into, inspect variables, use a **conditional breakpoint** ("only when amount == 0"). Learn it in your IDE this week.

### 13.5 Logging

Levels: `ERROR` , `WARN` , `INFO` , `DEBUG` , `TRACE` . Log **facts with identifiers** ("transfer {} failed: {}"), not secrets or full personal data. Use a correlation id so one request can be followed across logs.

### 13.6 Docker (basics)

An image is a packaged filesystem + command; a container is a running image. Learn `Dockerfile` , `docker run` , `docker compose` . Databases in containers are how you run realistic tests locally.

### 13.7 Continuous integration

On every push a server builds and runs the tests (CI). A red build blocks merging. This is why tests must be deterministic.

---

## 14. Production engineering

---

### 14.1 Observability

- **Logs** (what happened), **metrics** (counts, latencies; Prometheus), **traces** (a request across services).

- **Health probes:** *liveness* (restart me?) vs *readiness* (send me traffic?).
- Alert on symptoms users feel (error rate, latency), not only on CPU.

## 14.2 Configuration

Settings (limits, fees, URLs, credentials) live outside the code: environment variables and config files per environment. Validate configuration at startup and **fail fast**. **Bug you met:** an operational endpoint that was simply not listed in the config.

## 14.3 Security basics (learn the OWASP Top 10)

- Never trust input: validate, bound, parameterize SQL (no string concatenation).
- Authentication (who are you) vs authorization (what may you do). Check on the server, always.
- Do not put secrets in code or logs; use environment variables / a secrets manager.
- Do not expose internals in error messages.
- Keep dependencies updated.

## 14.4 Reliability patterns

Timeouts on every external call; retries with backoff **and idempotency**; circuit breakers; graceful shutdown (finish in-flight requests); rate limiting.

## 14.5 Data safety

Money and other critical data: transactions, constraints, an audit trail (the ledger), backups, and never editing history: add correcting entries instead.

## 14.6 Working in a team

Read code more than you write it. Small pull requests, clear descriptions, respond to review without ego. Write down decisions. Ask for help after you have tried things and can say what you tried.

---

# 15. A study plan

---

Roughly 10-12 weeks part-time; adjust to your pace. Every week finish with a small program of your own.

| Weeks | Topic                                                                                        | Do this                                                                                                |
|-------|----------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|
| 1-2   | Java core: types, control flow, classes, <code>==</code> / <code>equals</code> , collections | Re-do <code>LibraryManagementSystem</code> from scratch; explain every fix aloud                       |
| 2-3   | Git, terminal, debugger, reading stack traces                                                | Put your projects in Git; debug one failing test only with the debugger                                |
| 3-4   | Numbers/money, exceptions, <code>java.time</code>                                            | Write <code>FeePolicy</code> and a daily-schedule class yourself with tests                            |
| 4-5   | Data structures and algorithms (§12)                                                         | Solve 3-4 easy problems a week (lists, maps, recursion, graphs)                                        |
| 5-6   | SQL and databases (§7.1-7.5)                                                                 | Model wallets/transfers in plain SQL; write the joins by hand                                          |
| 6-7   | JPA/Hibernate, transactions                                                                  | Re-read <code>WalletService</code> : predict each <code>@Transactional</code> behaviour before running |
| 7-8   | HTTP/REST, validation, error handling (§8)                                                   | Add one endpoint with validation and proper status codes                                               |
| 8-9   | Testing (§10)                                                                                | Add 10 tests to a project, including boundaries and a failure path each                                |
| 9-10  | Concurrency (§6)                                                                             | Reproduce the lost update with plain threads, then fix it three ways (atomic, lock, DB)                |
| 10-12 | Design, production topics, security basics                                                   | Read <code>WalletService/reference</code> and write a one-page design note                             |

**Practice loop that works:** build small -> break it on purpose (plant your own bug) -> write the test that catches it -> fix -> explain the fix in writing. Planting bugs teaches you where bugs live.

## Suggested resources

- *Effective Java* (Joshua Bloch): the best next book after the basics (`equals`/`hashCode`, immutability, generics, exceptions).
- *Java Concurrency in Practice* (Goetz et al.): when you are ready for §6 in depth.
- *Designing Data-Intensive Applications* (Kleppmann): databases, transactions, distributed systems.
- *Clean Code, Refactoring* (Fowler): readable, changeable code.
- *The Pragmatic Programmer*: habits and mindset.
- Official docs: the Java `java.time`, `java.util.concurrent` and Spring guides; Use The Index, Luke (SQL indexing).
- Practice: LeetCode/Exercism for algorithms; write and break your own small services.

## 16. Glossary

- **Atomic:** happens as one indivisible step.
- **Boundary condition:** behaviour exactly at a limit (`0`, `max`, `>=` vs `>`).
- **Check-then-act (TOCTOU):** a decision made on data that changes before you act.

- **Correlation id:** an id carried through logs to follow one request.
- **DTO:** data transfer object; the shape of an API request/response.
- **Deadlock:** threads/transactions each waiting for the other's lock.
- **Defensive copy:** copying a mutable object so callers cannot change your internal state.
- **Dirty checking:** the ORM detecting changed fields and writing them at flush.
- **Flush:** writing pending ORM changes to the database.
- **Idempotent:** doing it many times has the same effect as once.
- **Invariant:** a condition that must always hold (balance == sum of ledger entries).
- **Isolation level:** how much concurrent transactions can see of each other.
- **Ledger:** append-only record of every balance change.
- **Lost update:** two writers read the same value and one overwrites the other.
- **Minor units:** the smallest currency unit (cents) used to store money as integers.
- **MDC:** per-thread logging context.
- **Optimistic / pessimistic locking:** detect conflicts at write time / prevent them with locks.
- **Persistence context:** the ORM's per-transaction cache of loaded entities.
- **Race condition:** result depends on the timing of concurrent operations.
- **Regression:** something that used to work and broke.
- **Stack trace:** the chain of calls that led to an error.
- **Thread-safe:** correct when used by several threads at once.
- **Transaction:** a group of operations that succeed or fail together.